# Evaluación de la Unidad 2

Diseño e implementación de un backend seguro completo
Fecha de Entrega: 09 de noviembre del 2026

### Instrucciones

En los mismos equipos que desarrollaron el frontend, diseñen e implementen un backend seguro completo para el caso de negocio que eligieron entre los 40 casos del curso. Utilicen las pantallas y los flujos de su diseño responsivo como punto de partida y mantengan la coherencia con los requisitos del caso.

Su implementación debe identificar y desarrollar los servicios para agregar, modificar, eliminar y consultar información; modelar las clases y sus cardinalidades; definir al menos los sistemas inventory, orders, clients y payments; asignar permisos HTTP a los stakeholders; diseñar el o los API Gateway; documentar sus rutas y secretos de integración; justificar la protección de los datos sensibles y de los secretos asociados al cliente; separar los servicios privados por su naturaleza; implementar agregaciones mediante GraphQL Gateway; y utilizar gRPC y servicios de analítica cuando las necesidades del caso lo justifiquen.

Completen los diez módulos de esta guía y entreguen las evidencias indicadas en cada uno. Fundamenten las decisiones con reglas del negocio y criterios de seguridad. Los ejemplos orientan el trabajo y no deben considerarse una solución completa ni sustituir el diseño propio del equipo.

Modalidad: trabajo en equipo. Producto evaluado: backend completo integrado con el frontend, código fuente, informe de arquitectura y evidencias de ejecución y pruebas. Puntaje total: 100 puntos. La fecha y el medio de entrega serán comunicados por el docente.

### Propósito de la evaluación

Diseñar, implementar y validar el backend completo del caso de negocio elegido por el equipo, integrado con el frontend responsivo desarrollado. La propuesta debe conectar cada interacción del frontend con servicios, sistemas, permisos, rutas de API Gateway y mecanismos de protección de datos y secretos.

La evaluación exige el diseño, la implementación completa y la validación del backend. Deben funcionar los requisitos del caso y los flujos del frontend, con persistencia, reglas de negocio, autenticación, autorización e integraciones. Los ejemplos son fragmentos orientativos; cada equipo debe tomar y fundamentar sus propias decisiones.

### Punto de partida

1. Identifiquen el número y nombre del caso elegido, integrantes y versión del frontend.
2. Reúnan las pantallas y flujos existentes para escritorio y móvil. Si una función del caso todavía no tiene pantalla, registren la brecha y propongan cómo se incorporará.
3. Usen el caso de negocio como fuente de requisitos. Distingan esos requisitos de las decisiones de arquitectura que deben justificar en esta evaluación.

### Convenciones que deberán aplicar

Dato público: información que puede divulgarse sin identificar ni perjudicar a una persona, por ejemplo la descripción de un producto. API pública: interfaz accesible por clientes externos, que

puede exigir autenticación. Servicio privado: servicio interno cuyo acceso se restringe a componentes autorizados. Son clasificaciones diferentes.

Para el inventario inicial se pueden asumir públicos los datos del catálogo. Si el frontend utiliza RUN, dirección, correo, historial de pedidos u otros datos personales, clasifíquenlos como restringidos y justifiquen su protección. Que aparezcan en una pantalla no los convierte en públicos.

Stakeholder significa parte interesada. Algunos stakeholders tendrán un rol que invoca servicios; otros solo reciben información o supervisan y deberán registrarse sin permisos operativos.

### Recorrido y entrega

Completen los diez módulos en orden. Mantengan identificadores consistentes de pantalla, requisito y servicio. Entreguen el backend ejecutable e integrado con el frontend, junto con un informe que contenga las matrices, los diagramas UML de clases, el mapa de sistemas y despliegue, los contratos y las evidencias de pruebas. Adjunten los diagramas en formato editable y legible.

Se exige implementar todos los flujos del frontend y los requisitos del caso; las brechas declaradas deberán resolverse en la entrega. La cantidad de endpoints debe responder a las necesidades del caso y a las decisiones de diseño del equipo.

# Módulo 1 Servicios derivados del frontend

## Propósito

Identificar qué operación necesita cada pantalla y qué datos atraviesan la frontera entre navegador y backend.

## Pasos a seguir

1. Recorran cada pantalla y anoten acciones, formularios, listados y mensajes esperados. Comparen escritorio y móvil: una variación visual no implica un servicio nuevo.
2. Asignen un identificador a cada servicio y vincúlenlo con una pantalla y un requisito del caso. Reutilicen servicios cuando varias pantallas ejecuten la misma operación.
3. Clasifiquen la operación como consultar, agregar, modificar o eliminar y propongan GET, POST, PUT o DELETE. No inventen operaciones destructivas para completar un CRUD.
4. Definan parámetros de ruta, consulta, cabeceras necesarias y cuerpo. Indiquen campos, tipos, obligatoriedad, validación y clasificación de cada dato.
5. Definan respuesta, códigos de estado y errores relevantes. Diferencien lo que envía el navegador de lo que calcula o valida el servidor.

## Qué deben completar

Una fila por operación: ID de servicio | requisito | pantalla y acción | propósito | verbo y ruta propuesta | entrada | salida | reglas y errores | clasificación de datos | acceso anónimo o autenticado. Para cada contrato adjunten un ejemplo JSON con datos ficticios.

## Ejemplo parcial

| Campo | Fragmento orientativo |
|---|---|
| Servicio | S01 Consultar catálogo |
| Pantalla y acción | P01 Catálogo -> cargar productos disponibles |
| Contrato | GET /api/v1/products?available=true |
| Entrada | available: boolean opcional. Sin cuerpo. |
| Salida | 200 con una lista de {id, nombre, precio, disponible}; [] si no hay resultados. |
| Regla | 400 ante filtros inválidos. Devolver solo campos permitidos del catálogo. |

El equipo debe definir los demás servicios y contratos. Los precios, totales y estados de pago enviados desde el navegador no constituyen evidencia confiable para el servidor.

## Control antes de avanzar

¿Cada acción tiene un servicio o una justificación? ¿La matriz distingue campos públicos de datos restringidos? ¿Los contratos explican qué se transporta, sin copiar todo el modelo de base de datos?

# Módulo 2 Clases y cardinalidades del dominio

## Propósito

Representar las entidades necesarias para las operaciones del frontend y los servicios que las exponen, con relaciones coherentes con el caso.

## Pasos a seguir

1. Identifiquen las clases del dominio a partir del caso, los formularios y los contratos. Incluyan atributos relevantes, tipos e identificadores.
2. Representen los servicios públicos como clases o interfaces con el estereotipo «service». Incluyan operaciones que correspondan a los IDs del módulo 1.
3. Relacionen entidades mediante asociaciones, composición o agregación cuando corresponda. Escriban las multiplicidades en ambos extremos: 1, 0..1, 0..* o 1..*.
4. Relacionen servicios y entidades con dependencias UML cuando el servicio usa una clase. No interpreten esas dependencias como relaciones entre registros de negocio.
5. Expliquen las reglas que originan cada cardinalidad y separen el modelo interno de los DTO que se envían al frontend.

## Qué deben completar

Diagrama UML de clases del dominio y los servicios públicos; leyenda de notación; tabla de relaciones con clase origen, clase destino, multiplicidades y justificación; correspondencia entre operación UML e ID de servicio.

## Ejemplo parcial

Pedido «1» – «1..*» DetallePedido. Bajo la regla de que un pedido confirmado contiene al menos un ítem, cada detalle pertenece a un pedido y cada pedido tiene uno o más detalles. Si el equipo modela un borrador vacío, deberá explicar cómo cambia esta regla.

CatalogService depende de ProductoDTO para devolver consultarCatalogo(filtro). Esa dependencia indica uso de un contrato, no que exista un producto por cada instancia del servicio.

## Decisiones que debe resolver el equipo

¿El carrito y el pedido son clases distintas? ¿Qué ocurre con el historial si un producto deja de estar disponible? ¿Qué atributos se conservan internamente y cuáles salen en cada respuesta? La guía no entrega el diagrama completo.

## Control antes de avanzar

¿Todas las operaciones del módulo 1 están representadas? ¿Cada asociación tiene ambas multiplicidades y una regla que las respalda? ¿El diagrama evita exponer datos internos solo porque forman parte de una clase?

# Módulo 3 Sistemas e integraciones

## Propósito

Distribuir responsabilidades entre sistemas y explicar cómo colaboran sin duplicar el control de los mismos datos.

## Pasos a seguir

1. Incluyan como mínimo inventory, orders, clients y payments. Implementen estos subsistemas como microservicios privados con contratos y responsabilidades definidos. Justifiquen sus límites y cualquier separación adicional según cohesión, acoplamiento y propiedad de datos; no dividan un servicio por cada tabla o pantalla.
2. Asignen una responsabilidad y los datos bajo autoridad de cada sistema. Identifiquen cuál decide el estado oficial de cada recurso.
3. Mapeen los servicios del módulo 1 a estos sistemas y documenten sus dependencias.
4. Definan el intercambio entre sistemas: operación o evento, datos mínimos, autenticación entre componentes y tratamiento de errores.
5. Agreguen sistemas complementarios exigidos por el caso, como despacho, boletas o mensajería. Distingan servicios propios de proveedores externos.

## Qué deben completar

| Sistema mínimo | Aspecto que deben definir |
|---|---|
| inventory | Catálogo, disponibilidad, stock o reservas, según el caso. |
| orders | Creación y ciclo de vida de pedidos; reglas de transición. |
| clients | Registro, perfil y acceso autorizado a datos de clientes. |
| payments | Inicio de pagos, comprobación del resultado y relación con el proveedor. |

Completen una matriz: sistema | responsabilidad | datos que administra | servicios asociados | dependencias | contrato de integración | credencial o identidad requerida | respuesta ante fallos. Añadan un mapa de componentes y una secuencia de un flujo representativo.

## Ejemplo parcial

orders consulta a inventory para verificar disponibilidad de un ítem antes de aceptarlo. El equipo debe decidir si necesita reservar stock, qué pasa si la respuesta no llega y cómo evita confirmar unidades que ya no están disponibles.

Para pagos, expliquen cómo se verifica el aviso del proveedor, cómo se evita procesar dos veces una misma notificación y qué sistema autoriza el cambio de estado. El frontend no debe decidir por sí mismo que un pedido está pagado.

### Control antes de avanzar

¿Los cuatro sistemas tienen funciones reales? ¿Cada dato tiene una autoridad definida? ¿La propuesta contempla fallos de comunicación y consistencia del negocio, además del camino exitoso?

# Módulo 4 Stakeholders y permisos HTTP

## Propósito

Definir quién necesita cada servicio, para qué lo utiliza y sobre qué recursos puede actuar.

## Pasos a seguir

1. Identifiquen stakeholders del caso y su interés. Separen partes interesadas, roles de usuario e identidades técnicas.
2. Relacionen cada rol operativo con servicios concretos del módulo 1.
3. Para cada servicio indiquen GET, POST, PUT y DELETE como permitido, denegado o no aplicable. Un verbo permitido no autoriza todas las rutas.
4. Agreguen condiciones: propiedad del recurso, ámbito, estado del pedido u otra regla. Definan autenticación y responsable de verificar la autorización.
5. Registren stakeholders sin acceso directo con la indicación "sin permisos operativos" y expliquen cómo reciben la información que necesitan.

## Qué deben completar

Registro de stakeholders: nombre o categoría | interés | rol asociado | uso de información. Matriz de permisos: rol o identidad | ID de servicio | ruta | verbo | decisión | alcance y condición | lugar donde se verifica.

## Ejemplo parcial

| Actor | Operación | Permiso y condición |
|---|---|---|
| Visitante | GET del catálogo público | Permitido para los campos públicos. |
| Cliente autenticado | GET de un pedido | Permitido solo si el pedido le pertenece; el backend verifica la propiedad. |
| Cliente autenticado | DELETE de un producto | Denegado; consultar el catálogo no otorga permisos de administración. |

Estas filas son ejemplos de permisos, no un inventario completo de roles. El equipo deberá fundamentar los permisos de administración y las identidades de proveedores o sistemas.

## Criterios técnicos

GET consulta; POST crea recursos o ejecuta operaciones justificadas; PUT reemplaza la representación del recurso indicado; DELETE solicita su eliminación según las reglas del negocio. Si necesitan actualización parcial, pueden proponer PATCH como complemento y explicar su semántica.

Una anulación de pedido no equivale necesariamente a borrar el registro. Justifiquen cómo se conserva el historial. Ocultar un botón en el frontend no sustituye el control de permisos del backend.

### Control antes de avanzar

¿Los permisos se expresan por servicio y recurso? ¿Se distinguen identidad y autorización? ¿La matriz permite comprobar que un usuario no consulta pedidos de otro?

# Módulo 5 Clases del API Gateway

## Propósito

Diseñar e implementar el o los API Gateway y sus políticas para separar la interfaz externa de los servicios internos, y representar los GraphQL Gateway dentro de la red privada.

## Pasos a seguir

1. Decidan cuántos gateways necesitan y por qué. Un gateway puede separar rutas mediante políticas; usar varios requiere una justificación concreta.
2. Clasifiquen rutas externas anónimas, externas autenticadas, administrativas e internas. Una ruta pública de red puede transportar datos restringidos para un usuario autorizado.
3. Modelen clases o interfaces como ApiGateway, RouteDefinition, AuthenticationPolicy, AuthorizationPolicy, ServiceTarget y SecretProvider. Añadan GraphQLGateway, Schema, Resolver y adaptadores hacia subsistemas; incluyan clientes gRPC y servicios especializados cuando correspondan. Adapten los nombres a su propuesta.
4. Incluyan atributos y operaciones relevantes y multiplicidades estructurales. Identifiquen dependencias hacia autenticación, sistemas internos y proveedor de secretos.
5. Expliquen qué valida el gateway y qué valida cada sistema. Los servicios internos deben rechazar llamadas no autorizadas y conservar las comprobaciones de negocio y propiedad.

## Qué deben completar

Segundo diagrama UML de clases: gateway y políticas, relaciones, multiplicidades y dependencias. Añadan una explicación de las fronteras de confianza y un recorrido breve de una solicitud autorizada y una denegada. El mapa de componentes del módulo 3 complementa este UML, no lo reemplaza. Identifiquen API Gateway, Auth Service, administrador de secretos, GraphQL Gateway, backend privado y, cuando proceda, servicios de analítica y de alta demanda. Distingan relaciones estructurales UML de flujos de red y dependencias de comunicación.

## Ejemplo parcial

ApiGateway «1» – «1..*» RouteDefinition, suponiendo que cada definición pertenece a un gateway. ApiGateway depende de SecretProvider para obtener una credencial necesaria al comunicarse con un destino interno. Si comparten políticas entre rutas, deberán modelar y justificar esa relación.

## Uso del material de apoyo

El ZIP del curso contiene un servicio de autenticación, un gateway y un backend de ejemplo. Úsenlo para reconocer responsabilidades y flujo de llamadas. Su existencia no demuestra que estén resueltas las políticas de cada caso ni la separación de inventory, orders, clients y payments.

La propuesta debe considerar validación de identidad, autorización por ruta, límites de solicitudes, tiempos de espera, mensajes de error controlados y registros sin secretos. Una redirección genérica debe restringir explícitamente los destinos, rutas y verbos admitidos.

### Control antes de avanzar

¿Se entiende cómo se impide el acceso directo no autorizado a los sistemas internos? ¿El diagrama distingue configuración de rutas, políticas y secretos? ¿La elección de uno o varios gateways tiene fundamento?

# Módulo 6 Rutas y secretos de integración

## Propósito

Documentar cómo cada solicitud externa llega al sistema correspondiente y qué identidad o secreto se utiliza en cada tramo.

## Pasos a seguir

1. Construyan una lista explícita de rutas y verbos permitidos. Vinculen cada fila con un servicio y una política del módulo 4.
2. Indiquen ruta externa, transformación necesaria y ruta interna de destino. Eviten publicar endpoints internos por omisión.
3. Distingan la credencial del usuario de la credencial entre sistemas. Documenten cómo se conserva una identidad confiable cuando el backend necesita comprobar propiedad.
4. Para cada integración indiquen si usa secreto, certificado o identidad de servicio. Si no requiere secreto estático, justifiquen el mecanismo.
5. Definan responsable, almacenamiento, permisos mínimos, rotación y revocación. Documenten referencias simbólicas, nunca valores reales.

## Qué deben completar

Matriz de rutas: ID | gateway | verbo | ruta externa | acceso y rol | sistema y ruta destino | controles | referencia de credencial. Para GraphQL añadan operación, tipo Query o Mutation, campos autorizados, resolver y subsistemas consultados. Para gRPC añadan servicio y método, mensaje de entrada y salida, modalidad unary o streaming, identidad permitida y plazo máximo. POST /graphql no significa que todas sus operaciones estén autorizadas: controlen cada operación, recurso y campo. Inventario de secretos: referencia | tipo | emisor | consumidor | finalidad | ubicación segura | alcance | rotación y revocación.

## Ejemplo parcial

S01 | Gateway externo | GET /api/v1/products | anónimo | inventory: GET /products | validación de filtros y límite de solicitudes | referencia inventory-read, si se elige una credencial para ese tramo.

Referencia inventory-read | credencial de servicio | consumidor gateway | finalidad consultar catálogo interno | permisos solo lectura | almacenamiento en gestor de secretos. El equipo debe definir emisión, expiración y reemplazo; no incluir la clave en el informe.

## Decisiones de seguridad

No asignen el mismo secreto a todos los sistemas sin justificarlo. Una credencial de payments debe tener únicamente los permisos necesarios para esa integración. La credencial de base de datos pertenece al sistema que accede a ella, no al navegador ni necesariamente al gateway.

El gateway debe descartar cabeceras de identidad interna aportadas por el navegador y transmitir únicamente contexto verificado. Eviten secretos en rutas, parámetros de consulta, repositorios, capturas y logs. Utilicen TLS en las comunicaciones que transportan credenciales o datos restringidos.

### Control antes de avanzar

¿Cada ruta tiene destino, autorización y controles definidos? ¿Cada credencial tiene un consumidor y una finalidad? ¿Está previsto qué hacer si una clave se filtra o el gestor de secretos no responde?

## Módulo 7 Datos sensibles y secretos del cliente

### Propósito

Decidir qué información debe resguardarse y evitar confundir datos personales, credenciales del usuario y secretos de integración.

### Pasos a seguir

1. Inventaríen campos públicos, datos personales, datos sensibles según el contexto y credenciales. Expliquen por qué se necesita cada campo.
2. Determinen si su propuesta necesita custodiar secretos asociados al cliente. Respondan sí o no y fundamenten; no supongan que todo dato restringido es una clave.
3. Definan quién puede consultar o modificar cada dato, qué campos devuelve cada servicio y cómo se limita el acceso a los registros del propio usuario.
4. Indiquen medidas de almacenamiento y transporte, retención, eliminación y exclusión de logs. Si gestionan contraseñas, propongan hash adaptativo con sal, no cifrado reversible.
5. Expliquen el manejo de sesiones y tokens, su expiración y revocación. Un navegador no puede mantener confidencial una clave compartida incluida en su código.

### Qué deben completar

Matriz: dato o credencial | finalidad | clasificación | sistema custodio | roles autorizados | representación en el frontend | protección | retención. Añadan una decisión explícita sobre secretos del cliente y un recorrido de un dato restringido desde su captura hasta su eliminación o conservación justificada.

### Ejemplo parcial

| Elemento | Tratamiento orientativo |
|---|---|
| Dirección de entrega | Dato personal. Mostrar únicamente a actores autorizados según el pedido; no incluirla en el catálogo público. |
| Clave del proveedor de pagos | Secreto de integración. Reside en el backend o gestor de secretos; el navegador no debe recibirla. |
| Contraseña del usuario | Credencial. Si la aplicación la administra, conservar un hash adaptativo con sal y no devolverla en respuestas. |

Si el proveedor admite un token de pago o una clave publicable, identifiquen su alcance y su clasificación a partir de su contrato. No confundan esos identificadores con una clave secreta que permite operar la cuenta del comercio.

### Casos de validación que deben proponer

Documenten entrada, identidad, resultado esperado y componente que lo verifica: acceso anónimo a catálogo; operación protegida sin sesión válida; operación de administración desde un rol sin permiso; consulta de un pedido ajeno; intento de acceso directo al backend sin identidad autorizada; intento

de confirmar pago mediante datos alterados del frontend. Ejecuten estas pruebas contra el backend implementado y registren resultados verificables. El análisis del diseño por sí solo no sustituye la prueba de ejecución.

### Control antes de entregar

¿Existe algún secreto en JavaScript, HTML, capturas o ejemplos? ¿Los datos retornados son los mínimos necesarios? ¿La propuesta protege información del cliente aunque la pantalla sea accesible por Internet?

# Módulo 8 Separación privada y agregaciones GraphQL

## Propósito

Organizar los servicios privados por su naturaleza y entregar al frontend agregaciones que combinen información de varios subsistemas sin exponerlos directamente.

## Pasos a seguir

1. Después del enrutamiento del API Gateway, distingan servicios transaccionales, agregación, analítica y alta demanda. Mantengan los servicios privados y el administrador de secretos sin acceso público directo.
2. Definan los límites de inventory, orders, clients y payments mediante cohesión, acoplamiento, mantenibilidad, evolución y propiedad de datos. Cada servicio modifica únicamente los datos bajo su responsabilidad; las colaboraciones utilizan contratos explícitos.
3. Implementen al menos un GraphQL Gateway privado y una agregación útil que combine dos o más subsistemas del caso. Documenten el o los gateways adicionales si los necesitan. No se exige federación: pueden resolver la composición mediante llamadas a APIs internas.
4. Definan schema, types, queries, resolvers y mutations cuando correspondan. Las reglas transaccionales permanecen en el servicio de negocio. La agregación no autoriza acceder directamente a bases de datos de otros servicios.
5. Apliquen identidad verificada, autorización por recurso y campo, paginación, límites de profundidad y costo de consultas, plazos máximos y tratamiento de errores parciales. Eviten llamadas repetidas por cada ítem mediante batching u otra estrategia justificada.
6. Integren la agregación con una pantalla real del frontend a través del API Gateway. Demuestren que el navegador no puede invocar directamente los subsistemas privados ni obtener campos que su rol no autoriza.

## Qué deben completar

Mapa de clasificación: servicio | naturaleza | responsabilidad | datos propios | contrato | dependencias | justificación de separación. Entreguen el schema GraphQL, resolvers, consulta de prueba, respuesta ficticia esperada, matriz de autorización y evidencia de la agregación ejecutada desde el frontend.

## Ejemplo parcial

Una consulta resumenPedido puede combinar el estado administrado por orders con descripciones públicas de productos de inventory. El equipo debe definir su schema, la autorización del pedido, los campos permitidos y qué sucede si inventory no responde. Este fragmento no define el circuito completo de compra.

## Control antes de avanzar

¿La consulta combina realmente subsistemas? ¿Se conserva la autoridad de cada servicio? ¿Las políticas impiden filtrar información restringida mediante una selección de campos GraphQL? ¿La red privada y los permisos se verifican en ejecución?

# Módulo 9 Analítica y comunicación gRPC cuando correspondan

## Propósito

Separar el procesamiento analítico y las cargas de alta demanda, eligiendo contratos y mecanismos de comunicación apropiados para las necesidades del caso.

## Pasos a seguir

1. Caractericen volumen de datos, concurrencia, tamaño de mensajes, frecuencia, latencia esperada y necesidad de streaming. Usen una carga de prueba reproducible y declaren si representa una proyección o una medición. No supongan que toda consulta de reporte necesita big data.
2. Decidan si se necesita un servicio de analítica y si existen operaciones que requieren comunicación interna gRPC. Documenten la decisión en un ADR con contexto, alternativas, elección y consecuencias. Si el caso requiere procesamiento intensivo o streaming y el contrato de comunicación justifica gRPC, deben implementarlo.
3. Si corresponde analítica, implementen un servicio privado separado de las transacciones y definan origen de datos, actualización, retraso aceptado, agregaciones, permisos y tratamiento de datos personales. Indiquen cómo el GraphQL Gateway consulta sus resultados cuando una pantalla lo necesita.
4. Si corresponde gRPC, implementen contratos .proto, mensajes tipados, servidor y cliente interno. Elijan llamada unary, streaming del servidor, del cliente o bidireccional según el flujo. Definan autenticación de servicio, TLS o mTLS, autorización por método, deadlines, cancelación, estados de error e idempotencia cuando sea necesaria.
5. Para streaming definan control de flujo, límites de mensajes y memoria, desconexiones, reconexión y recuperación sin duplicar efectos. Para operaciones largas justifiquen si necesitan un trabajo asíncrono con identificador, en lugar de mantener una RPC bloqueada.
6. Ejecuten una prueba de carga acorde con la decisión. Registren entorno, datos, concurrencia, duración, latencia p95, throughput, errores y consumo de CPU y memoria. Comparen los resultados con los objetivos declarados; no afirmen una mejora de rendimiento sin medición.

## Qué deben completar

Matriz de necesidades y ADR de aplicabilidad. Si aplican, código de analítica, contratos .proto, cliente y servidor gRPC, pruebas de integración y reporte de carga. Si no aplican, justifiquen explícitamente la ausencia de cada componente con los requisitos y la carga prevista. El backend obligatorio y la agregación GraphQL siguen siendo exigibles.

## Ejemplo parcial

Un servicio de monitoreo de despacho podría publicar eventos de estado mediante streaming de servidor gRPC hacia un consumidor privado. El equipo debe decidir si su caso necesita ese flujo y cómo recupera eventos tras una desconexión. Un reporte diario pequeño puede resolverse sin un subsistema de big data.

### Criterio técnico

gRPC aporta comunicación tipada y modalidades de streaming; no ejecuta por sí mismo el procesamiento de big data ni reduce automáticamente su costo de CPU o memoria. El procesamiento pertenece al servicio especializado. GraphQL puede combinar resultados de servicios REST o de adaptadores gRPC. Si el frontend debe recibir eventos, definan un transporte compatible con el navegador y su punto de adaptación, sin exponer gRPC interno directamente.

### Control antes de avanzar

¿La decisión responde a una necesidad medida o proyectada de forma explícita? ¿Están implementados los componentes que la propuesta declara necesarios? ¿Las pruebas muestran cómo se comportan ante carga, fallos y desconexiones?

# Módulo 10 Backend completo e integración verificable

## Propósito

Demostrar que la arquitectura funciona como una aplicación completa y que sus controles se cumplen durante la ejecución.

## Pasos a seguir

1. Implementen todos los requisitos funcionales del caso y las operaciones de sus pantallas: registro y autenticación, catálogo e inventario, pedidos, pagos, administración y los flujos complementarios de despacho, boletas, mensajería o reportes que correspondan. Conserven las restricciones tecnológicas vigentes del curso.

2. Implementen persistencia real, esquema y migraciones o scripts de creación, datos de prueba ficticios, validaciones del servidor, permisos, control de estados y manejo de errores. No se considera completo un backend que solo devuelve respuestas fijas o simula todas las operaciones en memoria.

3. Integren API Gateway, Auth Service, administrador de secretos, GraphQL Gateway y microservicios privados. Limiten los puertos expuestos, propaguen identidad verificada y mantengan secretos fuera del código y del navegador. Puede demostrarse la separación pública y privada mediante redes de contenedores u otro entorno reproducible; no se exige una cuenta de nube pagada.

4. Integren el frontend responsivo con las APIs reales. Para pagos y mensajería utilicen entornos de prueba del proveedor cuando estén disponibles. Si una integración externa no es accesible, documenten la limitación y proporcionen un adaptador de prueba que preserve contrato, autenticación y transiciones; esto no sustituye la implementación de los servicios propios.

5. Ejecuten pruebas de contratos, integraciones, reglas críticas de negocio y seguridad. Comprueben propiedad de recursos, roles, rechazo del acceso directo, límites GraphQL, indisponibilidad de dependencias, pagos duplicados y recuperación de stock cuando corresponda. Agreguen las pruebas gRPC y analíticas que resulten aplicables.

6. Entreguen instrucciones para iniciar y detener el sistema, configurar referencias de secretos, crear datos de prueba y ejecutar las pruebas. Incluyan dependencias versionadas, variables de configuración sin valores reales, health checks y logs con identificador de correlación sin credenciales ni datos personales innecesarios.

## Qué deben completar

Código fuente del frontend y backend, archivo de despliegue reproducible o scripts equivalentes, documentación de inicio, persistencia, contratos REST y GraphQL, contratos gRPC cuando correspondan, pruebas automatizadas y resultados. Incorporen una matriz requisito | pantalla | servicio | prueba | resultado | evidencia y una demostración de los flujos principales.

## Ejemplo parcial

Para una consulta protegida de pedido, muestren una ejecución autorizada y otra con una cuenta distinta que resulte denegada sin devolver sus datos. Relacionen ambas evidencias con la misma regla de autorización y el servicio que la aplica. El equipo debe completar las pruebas de los demás flujos.

### Condición de completitud

El informe y los diagramas deben describir la implementación entregada. La entrega documental por sí sola no cumple la consigna; tampoco la cumple un sistema con credenciales incrustadas o que omita un subsistema necesario del caso. Las evidencias de cada módulo deben corresponder a componentes y pruebas ejecutables.

## Entrega y pauta de evaluación

### Lista de entrega del equipo

1. Portada con caso, integrantes y versión del frontend; requisitos y brechas declaradas.
2. Módulo 1: trazabilidad de pantallas, catálogo completo de servicios y ejemplos de contratos.
3. Módulo 2: UML del dominio y servicios públicos, cardinalidades y justificaciones.
4. Módulo 3: matriz de sistemas, mapa de componentes y secuencia de integración.
5. Módulo 4: stakeholders y matriz de permisos por ruta, verbo y condición.
6. Módulo 5: UML del gateway y políticas, fronteras de confianza y decisiones de separación.
7. Módulo 6: matriz de rutas e inventario simbólico de secretos e identidades, operaciones GraphQL y métodos gRPC cuando correspondan.
8. Módulo 7: clasificación y protección de datos, decisión sobre secretos del cliente y resultados de pruebas.
9. Módulo 8: separación privada, schema y resolvers GraphQL, agregación funcional y evidencia desde el frontend.
10. Módulo 9: ADR de aplicabilidad, analítica y gRPC implementados cuando correspondan, y mediciones de carga.
11. Módulo 10: código fuente, persistencia, despliegue reproducible, integración completa, documentación y pruebas ejecutadas.

### Rúbrica de evaluación sobre 100 puntos

| Criterio | Puntos | Evidencia para logro completo |
|---|---|---|
| Servicios y contratos | 10 | Operaciones completas, trazabilidad y contratos coherentes con el frontend. |
| UML y límites de microservicios | 10 | Clases, cardinalidades y separación por naturaleza justificadas. |
| Sistemas y persistencia | 10 | inventory, orders, clients y payments funcionales, con datos persistentes y reglas de negocio. |
| Stakeholders y seguridad | 10 | Autenticación, permisos por recurso y protección de datos comprobadas. |
| API Gateway y secretos | 10 | Enrutamiento real, separación de red y gestión segura de credenciales. |
| Agregaciones GraphQL | 15 | Gateway privado con agregación de al menos dos subsistemas, autorización y límites probados. |
| Analítica y gRPC según aplicabilidad | 10 | Decisiones fundamentadas e implementación y pruebas de los componentes necesarios. |
| Backend completo e integración | 15 | Flujos del caso funcionales e integrados con el frontend responsivo. |
| Pruebas y reproducibilidad | 10 | Pruebas ejecutables, evidencias trazables y arranque documentado. |

Asignación por criterio: logrado = 100 % del puntaje; mayormente logrado = 75 %; parcial = 50 %; inicial = 25 %; ausente = 0 %. Valoren cobertura, corrección y consistencia entre módulos. Una afirmación de seguridad sin evidencia de ejecución no alcanza logro completo. En el criterio de analítica y gRPC se evalúan 4 puntos por caracterización de carga y ADR y 6 por implementación y pruebas cuando sean necesarios. Cuando la no aplicabilidad esté fundamentada, estos 6 puntos se evalúan mediante evidencia reproducible de que la alternativa implementada satisface la carga y los requisitos. Omitir la decisión o no implementar un componente necesario no equivale a no aplicabilidad.

Errores que deben corregirse: tratar todos los datos del navegador como públicos; otorgar permisos solo por ocultar botones; confiar en precios o pagos informados por el frontend; compartir secretos sin limitar su alcance; publicar servicios internos sin control; indicar cardinalidades sin regla de negocio.

### Fuentes y material de apoyo

Diagramas de referencia: Microservicios.drawio, criterios de separación y comunicación REST, GraphQL y gRPC; Integracion.drawio, relaciones de API Gateway, Auth Service, administrador de secretos, GraphQL Gateway y backend privado con analítica y alta demanda cuando correspondan. Adapten estos patrones al caso y justifiquen sus decisiones; no copien todas las tecnologías sin necesidad.

Material del curso: Innovación-1.pdf, listado de 40 casos; Caso19.docx, caso de referencia de ventas y delivery; apigateway.zip, ejemplo de autenticación, gateway y backend. Los ejemplos de esta guía son parciales y no resuelven ningún caso completo.

OWASP REST Security Cheat Sheet: `https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html`
OWASP Authorization Cheat Sheet: `https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html`
OWASP Secrets Management Cheat Sheet: `https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html`
Documentación oficial de seguridad GraphQL: `https://graphql.org/learn/security/`
Documentación oficial de gRPC y streaming: `https://grpc.io/docs/what-is-grpc/core-concepts/`
Documentación oficial de deadlines gRPC: `https://grpc.io/docs/guides/deadlines/`
Documentación oficial de control de flujo gRPC: `https://grpc.io/docs/guides/flow-control/`